COMP438/538 Reinforcement Learning

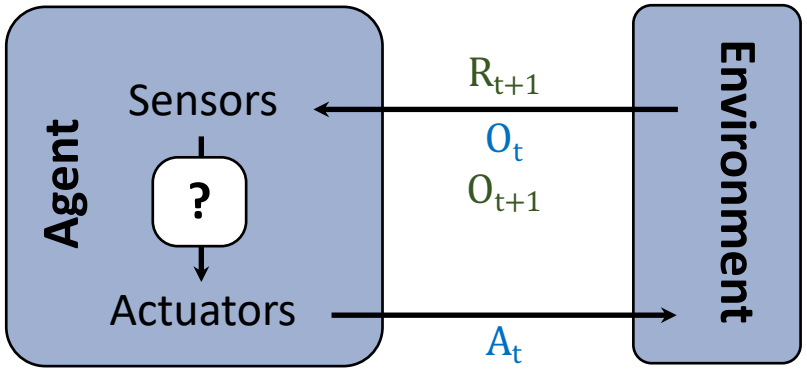
Markov Decision Processes

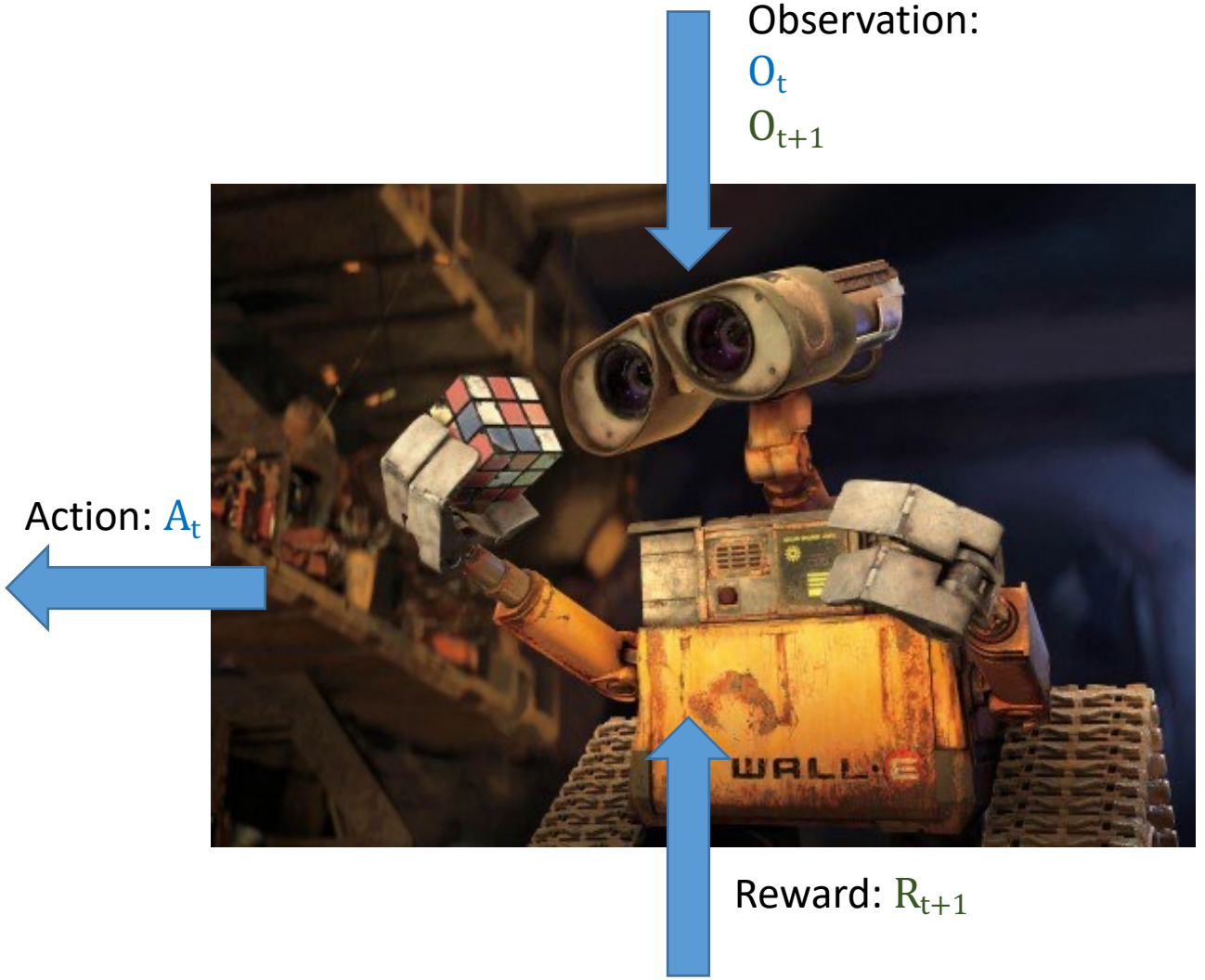
Barış Akgün

Fall 2026

The interaction loop

- Observe the environment: O_t
- Choose (and apply) an action: A_t
  - "Transition"
- Receive reward R_{t+1}
- Observe the environment: O_{t+1}
- …



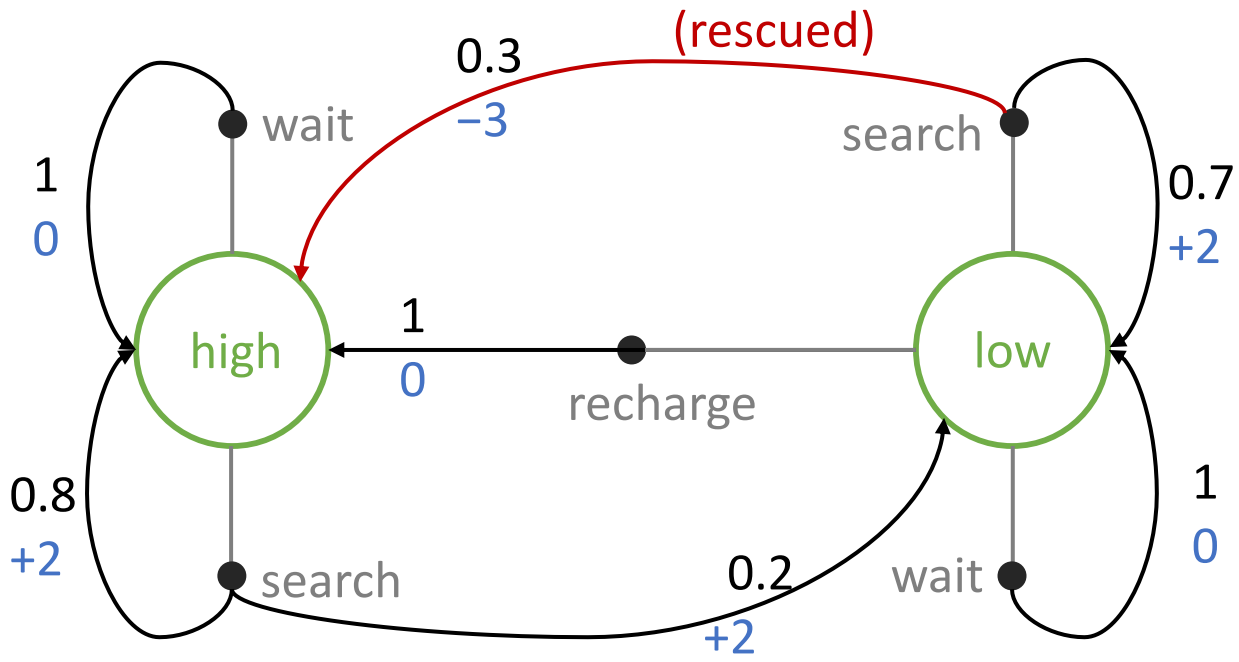


Food for thought:

- Where is the state?
- How do we choose the action?

The recycling robot

$$\mathcal{S} = \{\text{high}, \text{low}\}$$
$$\mathcal{A}(\text{high}) = \{\text{search}, \text{wait}\}$$
$$\mathcal{A}(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$$



Observation, history and state

- The loop leads to a sequence of observations, actions and rewards
- H_t: The information accumulated before choosing A_t.

$$H_t = (O_0, A_0, R_1, O_1, \dots, A_{t-1}, R_t, O_t), \qquad H_0 = (O_0)$$

- The environment state fully describes the environment:
  - Need not be visible to the agent (not everything is in the observations)
  - But might be indirectly estimated (e.g. no battery gauge in the recycling robot example)
  - Need not be relevant to the agent
  - Autonomous driving example: the car does not know the exact traffic, and the heights of the buildings do not matter
- Agent state is the internal representation used by the agent, built from the available history.

$$S_t = f(H_t)$$

- Ideally, the agent state holds enough information to make a decision (choose an action)

Transition

$$O_t, A_t \rightarrow R_{t+1}, O_{t+1}$$

- At each time step, the environment changes:
  - By the agent
  - By other agents
  - By its own dynamics
- This change can be deterministic or stochastic

- (Agent State) Transition Model/Dynamics: Mathematical model of how the agent's state evolves given agent actions
  - Mapping from S_t, A_t to R_{t+1}, S_{t+1}
  - Distribution over R_{t+1}, S_{t+1} given S_t, A_t
  - In most practical applications, "reward function" and "state transition model" are treated separately

Markov State Assumption and Models

"The future is independent of the past given the present" / "Future only depends on the current state"

$$S_{0:t-1} \rightarrow S_t \rightarrow S_{t+1:\infty}$$

- Simple state dynamics (no action):

$$\Pr\{ S_{t+1} \mid S_t, S_{t-1}, \ldots, S_0 \} = \Pr\{ S_{t+1} \mid S_t \}$$

- State dynamics with action:

$$\Pr\{ S_{t+1} \mid S_t, S_{t-1}, \ldots, S_0, A_t, A_{t-1}, \ldots, A_0 \} = \Pr\{ S_{t+1} \mid S_t, A_t \}$$

- Full dynamics: $\Pr\{ S_{t+1}, R_{t+1} \mid S_t, A_t \}$

- Markov Property and States?
  - $S_t = f(O_t)$ vs $S_t = f(O_{t-k+1:t})$ vs $S_t = H_t$

A chatbot's latest message

- Latest user message: **“Yes, please.”**
- Earlier question: “Would you like a shorter explanation?”
- Or: “Would you like a worked example?”
- **What earlier information would you keep to interpret “Yes, please” and choose a response?**

- Is using the last message as the state Markovian?
- Are chatbots Markovian?

Models Continued

- Predict what happens next given the current (Markov) state and an action
- Most generic model form:

$$\mathcal{M}(s, a, r, s') \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$

- State-only Markov Transition Model: Predicting the next state

$$\mathcal{T}(s, a, s') \doteq \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$$

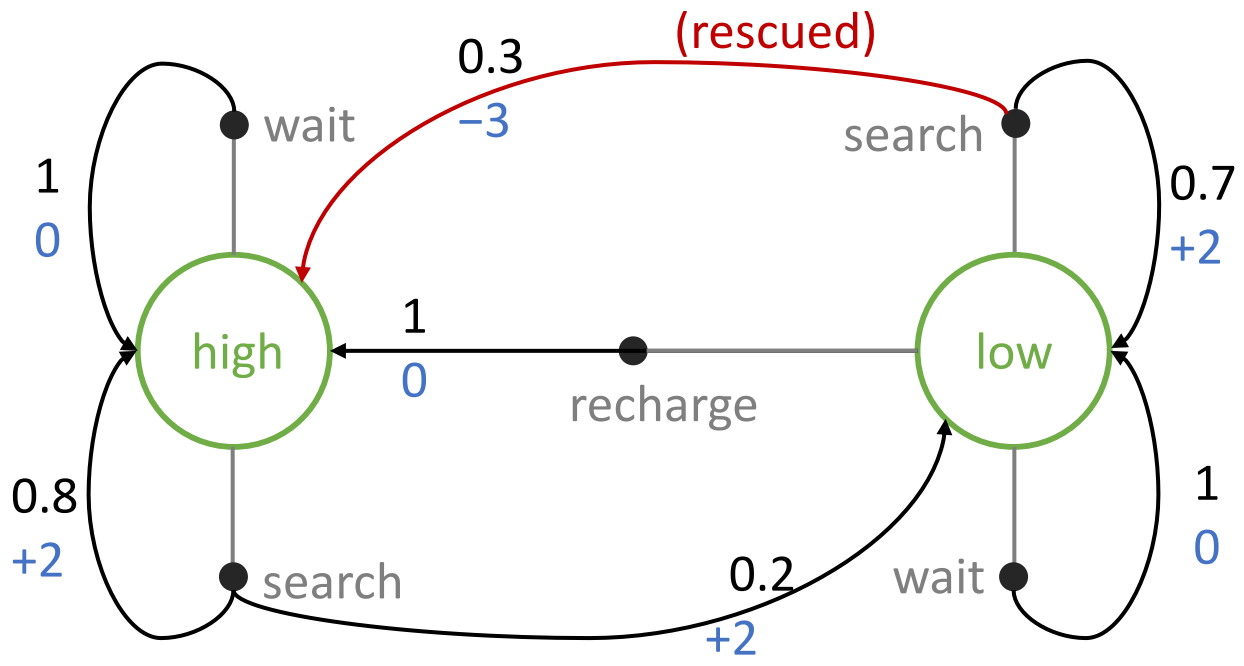
- Reward Model (or sometimes reward function): Predicting the immediate reward

$$\mathcal{R}(s, a) \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

- The reward function can have multiple forms:

$$\mathcal{R}(s), \mathcal{R}(s, a), \mathcal{R}(s, a, s'), \mathcal{R}(s')$$

Reading the robot's dynamics



s	a	r	s'	$\Pr\{s', r \mid s, a\}$
high	search	2	high	0.8
high	search	2	low	0.2
high	wait	0	high	1
low	search	2	low	0.7
low	search	−3	high	0.3
low	wait	0	low	1
low	recharge	0	high	1

- For each state/action pair, do the outcome probabilities add to one?

A reward outcome and an expected reward

$$\mathcal{T}(s, a, s') \doteq \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$$

$$\mathcal{R}(s, a) \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

- Searching at low charge gives **2** or **–3**.

$$\mathcal{R}(\text{low}, \text{search}) = 0.7(2) + 0.3(-3) = 0.5$$

- The expected reward is 0.5 but neither outcome gives 0.5.

Formalize Everything: Markov Decision Process

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$$

- In a Markov-state model:
  - At state S_t apply action A_t
  - Consequences are reward R_{t+1} and next state S_{t+1}
- A finite MDP specifies
  - State space: $\mathcal{S}$
  - Action space per state: $\mathcal{A}(s)$
  - Reward Model: $\mathcal{R}(\cdot)$
  - State dynamics:

    $\mathcal{T}(s, a, s') \doteq \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$

Or the full model:

$\mathcal{M}(s, a, r, s') \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$

- We will add more things soon

Policies

- Why are we doing all this?
- “Pick actions to maximize expected utility”

Policy: Mapping between states and actions

- A deterministic policy chooses an action: $a = \pi(s)$.
  - Search at high, recharge at low
- A stochastic policy assigns action probabilities: $\pi(a \mid s)$.
  - At low charge: search with probability 0.5, recharge with probability 0.5.

- **Policy probability:** which action the agent chooses.
- **Model probability:** what happens after that action.

Return

- How do we pick a policy?
- How do we know if a state is a “good state”?

- Remember the recycling robot:
  - Always search
  - Search when high, recharge when low
- Which action has higher reward for the low state?
- Which policy has a higher (expected) return?
- Reward: immediate, Return: Long-term
- How to calculate the return?
  - Sum?
  - Average?
  - Discounted sum?

Discounted return

- Given a sequence of transitions (potentially infinite) $\ldots S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}, \ldots$

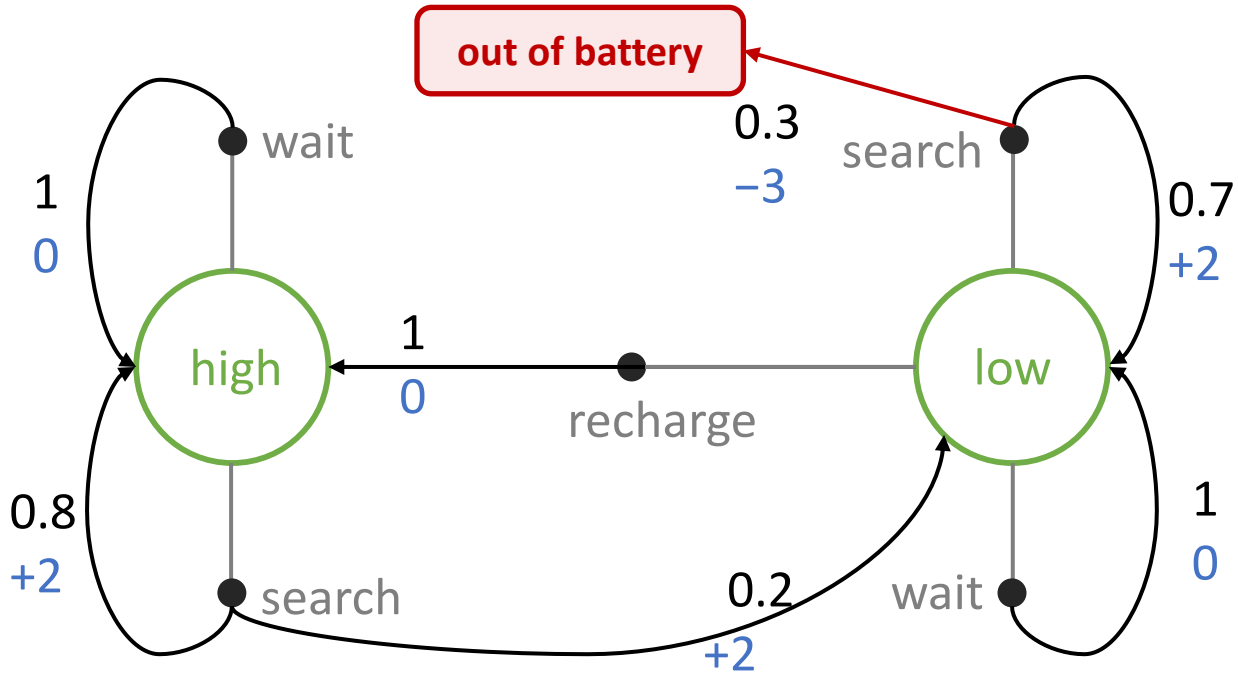
$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$G_t = R_{t+1} + \gamma G_{t+1}$$

- γ is called the discount factor
- For example, given $\gamma = 0.5,$ successive reward weights are 1, 0.5, 0.25, …
- Smaller γ gives less weight to later consequences.
- Bounded rewards and $0 \leq \gamma < 1$ keep the continuing sum finite.
- Important observation: Returns are calculated per time step and depend on the policy (but we don't have a single return number for a policy!)

One failure, two possible endings

- $\mathcal{S}^+$ includes the terminal state.
- After termination, imagine a self-loop producing reward **0** forever.
- A terminal state can exist even if some policies never reach it.



Model	Failed search at low charge
Continuing rescue	Reward −3; return to high charge and keep working
Terminal failure	Reward −3; the *episode* ends

Calculate the terminal return

The first three search transitions from our earlier example, now with a terminal ending:

Time t	State S_t	Action A_t	Reward R_{t+1}	Next state S_{t+1}
0	high	search	2	high
1	high	search	2	low
2	low	search	−3	terminal

$$T = 3, \qquad G_3 = 0, \qquad G_t = R_{t+1} + 0.5\, G_{t+1}$$

- Work backward: $G_2 = ?$ $G_1 = ?$ $G_0 = ?$

The continuing return has a tail

- Observed rewards: **2, 2, –3, 2**, then the robot continues.
- Four-step discounted reward sum:

$$2 + 0.5(2) + 0.5^2(-3) + 0.5^3(2) = 2.5$$

- Complete continuing return:

$$G_0 = 2.5 + 0.5^4 G_4$$

- These alone do not tell us G_4.

- Questions:
  - Is calculating a return for a single trajectory enough to differentiate policies?
  - What to do if the trajectory never terminates?

Values are expected returns

- Value of a state: Expected return when starting in state s, then following the policy π.

$$v^{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

- Value of a state-action pair (a.k.a. Q-value): Expected return when starting in s, taking action a, **then** following π.

$$q^{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

- A return belongs to a trajectory. A value averages possible returns under a policy.

A policy whose values we can calculate

- Let π **always wait**, at either charge level.
- Waiting preserves charge and gives reward 0.

$$G_t = 0 + 0.5(0) + 0.5^2(0) + \cdots = 0$$

$$v^\pi(\text{high}) = v^\pi(\text{low}) = 0$$

- What if we take one different action, then always wait?

The first action and the continuation policy

Take the specified action once, then always wait:

Starting state and first action	Expected complete return
high, search	$q^{\pi}(\text{high},\text{search})=2$
low, search	$q^{\pi}(\text{low},\text{search})=0.7(2)+0.3(-3)=0.5$
low, recharge	$q^{\pi}(\text{low},\text{recharge})=0$

- Why is recharging worth zero here, even though it can be useful?

Values of the two familiar policies

Continuing rescue model, $\gamma = 0.5$

Goal: find a policy that maximizes expected discounted return.

Policy	$v^{\pi}(\text{high})$	$v^{\pi}(\text{low})$
Always search	3.60	1.60
Search at high; recharge at low	3.64	1.82

- One observed excerpt cannot rank policies. These values average complete returns.

- But wait, how do we compute these values if the task continues forever?

Complete expected discounted returns, rounded to two decimals.

From returns to value computation

- For our robot:
  - What distinguishes a reward, a return and a value?
  - Which uncertainty belongs to the policy, and which to the model?
  - What is missing when we see only the first four rewards?
- **Next:** use the model to compute values (Bellman equations and policy evaluation).